

Data Engineering Project Report: Creator Listening Analytics

Repository: https://github.com/leeloaj/data_engineering_group5

Group Members & Contributions:

Villem Paabo, Markus Sulg, Karl Johannes Artma, Leelo Aino Jürgenstein, Kauri Remm had meetings, where we all contributed equally to the project report.

AI chat links:

- <https://share.gemini.google/i4HFPAtcQbd>
- <https://claude.ai/share/fe44a371-0661-4a2e-80d7-52b3e99568b6>
- <https://chatgpt.com/share/6ac3ed29-5aa4-83eb-a790-90d9fe18c41a>
- <https://claude.ai/share/99e5d40f-e98a-4f57-bbf3-e09dc3d47ccd>

1. Business Brief

Objective

Provide automated listening statistics and audio characteristic insights to creators (musicians/bands), enabling them to understand track performance over time and strategically curate music for target audiences.

Key Stakeholders

- Musicians and Independent Artists
- Bands and Content Creators
- Artist Management Teams & Record Labels

Key Metrics (KPIs)

1. **Total Streams:** Cumulative playback volume received by an artist's tracks over a specified date range.
2. **Average Track Popularity:** Mean popularity score (0–100) calculated across an artist's catalog.
3. **Stream Growth Rate:** Period-over-period percentage change in total streams.

Business Questions

- **BQ1:** Which of an artist's tracks receive the highest total streams?
- **BQ2:** How does an artist's track popularity change over time?
- **BQ3:** Which musical/audio characteristics (e.g., danceability, energy, valence) correlate with higher track popularity?
- **BQ4:** Which geographic regions or countries exhibit the highest streaming activity for an artist's catalog?
- **BQ5:** How does the streaming performance of an artist's new releases compare with their historical releases?

2. Datasets

Spotify Web API / Reccobeats API Data: Metadata (track ID, duration, explicit flag, popularity) merged with rich audio feature vectors (acousticness, danceability, energy, instrumentalness, key, liveness, loudness, mode, speechiness, tempo, valence).

Sources:

- Spotify API - <https://developer.spotify.com/documentation/web-api/reference/get-track>
- Reccobeats API (music characteristics) - <https://reccobeats.com/docs/apis/get-audio-features>

Spotify Charts Dataset (Kaggle / CSV Downloads): Historical global and regional daily listening/ranking metrics (track ID, country/region, chart position, daily streams, date).

Sources:

- Spotify Charts (recent day/week csv download) - charts.spotify.com
- Kaggle Spotify Charts data (2017 - 2021 historical metrics) <https://www.kaggle.com/datasets/dhruvildave/spotify-charts>

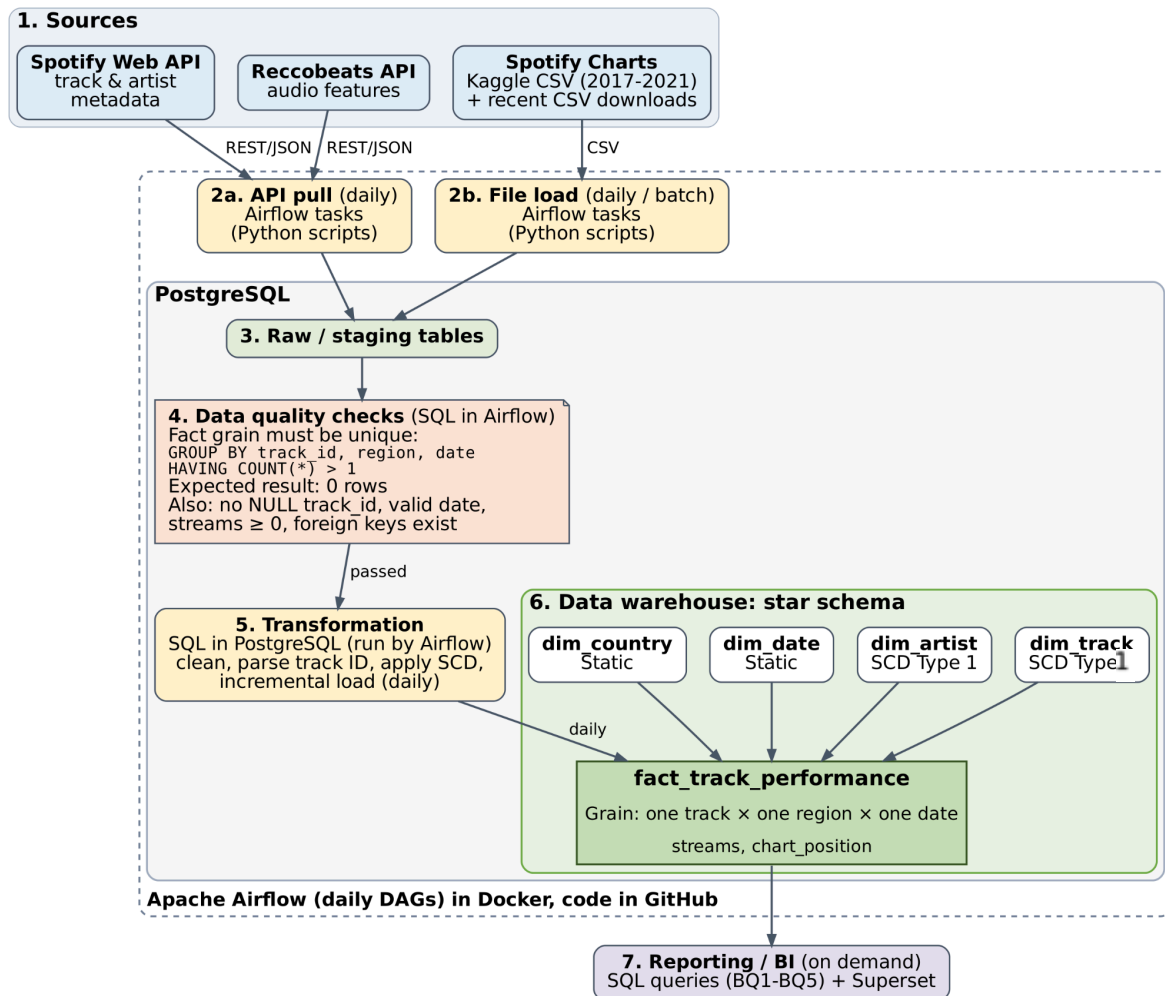
3. Tooling

Lifecycle stage	What happens in our project	Tool
Generation (sources)	Track and artist metadata, audio features, and historical chart/stream data	Spotify Web API, Reccobeats API, Spotify Charts CSV / Kaggle dataset
Ingestion	Daily API pull for metadata and audio features; daily or batch file load for chart CSVs	Airflow tasks (Python scripts)
Storage	Raw staging tables, then the warehouse (star schema)	PostgreSQL (raw/staging schema and warehouse schema)
Transformation	Clean and deduplicate, extract the track ID from the chart URL, apply SCD logic, load dimensions and fact table incrementally	SQL in PostgreSQL, run by Airflow
Serving	Analytical SQL and BI queries for the business questions	SQL (PostgreSQL), Superset (BI/dashboard tool)

Undercurrent	Tool
Orchestration	Apache Airflow (daily DAGs)
DataOps / version control	GitHub (DDL scripts, DAG definitions, documentation)
Reproducibility / deployment	Docker
Data quality / observability	SQL checks run as Airflow tasks
Data management / metadata	Data dictionary (this report)
Security	2FA on Github 😊

4. Data Architecture

The proposed architecture combines Spotify Web API metadata with historical Spotify chart/listening data. The Spotify Web API provides information about tracks, artists and albums but does not expose aggregate stream counts for tracks. The second dataset provides historical streaming performance.



Data Quality

Fact grain: one track × one region × one date.

An important check is the uniqueness of the fact-table grain.

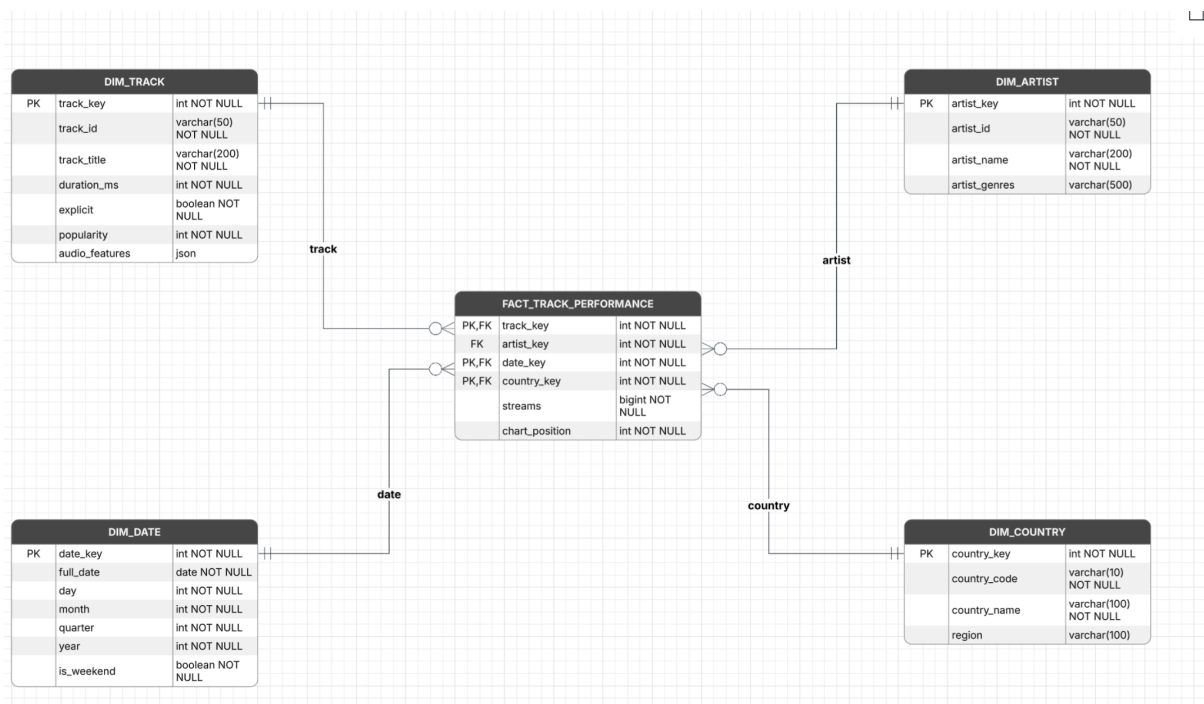
```
SELECT
  track_id,
  region,
  date,
  COUNT(*) AS record_count
FROM staging_performance
GROUP BY track_id, region, date
HAVING COUNT(*) > 1;
```

The expected result is **zero rows**.

Additional checks would include:

- **track_id** must not be NULL.
- **date** must be valid.
- **streams** must not be negative.
- Spotify track IDs should be unique in **dim_track**.
- Foreign keys in the fact table must exist in their corresponding dimensions.

5. Data Model



Grain Definition

- **FACT_TRACK_PERFORMANCE**: One record per track, per region, per calendar date

Dimension Classification & SCD Reasoning

1. **DIM_TRACK** (Type 1 SCD / Hybrid JSON):
 - Track title and duration are fixed (static attributes).
 - API popularity (0–100) updates in place via **SCD Type 1** to always display the latest global score without keeping historical table rows.
 - Detailed audio features (danceability, valence, etc.) are stored inside a JSON column **audio_features** for efficient attribute management.
2. **DIM_ARTIST** (Type 1 SCD):

- Over time, an artist's name or genres may be re-categorized. Changes overwrite existing rows directly (**SCD Type 1**) as historical tracking of minor genre renames is not required for core analytics.
3. **DIM_COUNTRY** (Static / Type 0):
 - Maps country codes (ISO 3166-1 alpha-2) to names and regional categories. ISO country definitions do not change over normal analytical lifecycles.
 4. **DIM_DATE** (Static / Type 0):
 - Pre-populated calendar table containing day, month, quarter, year, and weekend indicators.

6. Data Dictionary

Fact Table: **FACT_TRACK_PERFORMANCE**

Column Name	Data Type	Constraints	Description
track_key	INT	PK, FK, NOT NULL	Surrogate key linking to DIM_TRACK .
artist_key	INT	FK, NOT NULL	Surrogate key linking to DIM_ARTIST .
date_key	INT	PK, FK, NOT NULL	Foreign key referencing DIM_DATE (YYYYMMDD format).
country_key	INT	PK, FK, NOT NULL	Foreign key referencing DIM_COUNTRY .
streams	BIGINT	NOT NULL, >= 0	Total stream volume for the specified track, region, and date.
chart_position	INT	NOT NULL	Chart position on the specified date.

Dimension Tables

DIM_TRACK

Column Name	Data Type	Constraints	Description
track_key	INT	PK, NOT NULL	Auto-incremented surrogate key.
track_id	VARCHAR(50)	NOT NULL, UNIQUE	Spotify unique track identifier.
track_title	VARCHAR(200)	NOT NULL	Name of the track.
duration_ms	INT	NOT NULL	Duration in milliseconds.
explicit	BOOLEAN	NOT NULL	Flags whether track contains explicit lyrics.
popularity	INT	NOT NULL	Spotify popularity score (0–100).
audio_features	JSON	NULL	Nested JSON storing audio metrics (danceability, energy, valence, tempo, loudness, etc.).

DIM_ARTIST

Column Name	Data Type	Constraints	Description
artist_key	INT	PK, NOT NULL	Auto-incremented surrogate key.
artist_id	VARCHAR(50)	NOT NULL, UNIQUE	Unique Spotify artist identifier.
artist_name	VARCHAR(200)	NOT NULL	Name of the artist or band.
artist_genres	VARCHAR(500)	NULL	Comma-separated genre list.

DIM_DATE

Column Name	Data Type	Constraints	Description
date_key	INT	PK, NOT NULL	Date key formatted as YYYYMMDD.
full_date	DATE	NOT NULL	Standard calendar date.
day	INT	NOT NULL	Day of the month (1–31).
month	INT	NOT NULL	Month number (1–12).
quarter	INT	NOT NULL	Quarter number (1–4).
year	INT	NOT NULL	Four-digit calendar year.
is_weekend	BOOLEAN	NOT NULL	Boolean flag indicating weekend days.

DIM_COUNTRY

Column Name	Data Type	Constraints	Description
country_key	INT	PK, NOT NULL	Auto-incremented surrogate key.
country_code	VARCHAR(10)	NOT NULL	ISO 3166-1 alpha-2 country code.
country_name	VARCHAR(100)	NOT NULL	Full country name.
region	VARCHAR(100)	NULL	Geographical continent or region grouping.

7. Demo Queries

BQ1. Which of an artist's tracks receive the most streams?

```
SELECT
    t.track_id,
    t.track_title,
    SUM(f.streams) AS total_streams
FROM fact_track_performance f
JOIN dim_track t ON f.track_key = t.track_key
JOIN dim_artist a ON f.artist_key = a.artist_key
```

```

JOIN dim_country c ON f.country_key = c.country_key
WHERE a.artist_name = 'Arctic Monkeys'
  AND c.country_code <> 'GLOBAL'          -- avoid double counting global + regional
rows
  AND f.date_key BETWEEN 20170101 AND 20211231  -- optional date range
GROUP BY t.track_id, t.track_title
ORDER BY total_streams DESC
LIMIT 10;

```

BQ2. How does an artist's track popularity change over time?

```

SELECT
  d.year,
  d.month,
  t.track_id,
  t.track_title,
  t.popularity,          -- popularity at that time
  ROUND(AVG(f.chart_position), 1) AS avg_chart_position,
  SUM(f.streams)         AS monthly_streams
FROM fact_track_performance f
JOIN dim_track t ON f.track_key = t.track_key
JOIN dim_artist a ON f.artist_key = a.artist_key
JOIN dim_date d ON f.date_key = d.date_key
WHERE a.artist_name = 'Arctic Monkeys'
GROUP BY d.year, d.month, t.track_id, t.track_title, t.popularity
ORDER BY t.track_title, d.year, d.month;

```

BQ3. Which musical characteristics are associated with higher track popularity?

```

WITH track_features AS (
  SELECT
    t.track_id,
    t.popularity,
    (t.audio_features->'danceability')::FLOAT AS danceability,
    (t.audio_features->'energy')::FLOAT      AS energy,
    (t.audio_features->'valence')::FLOAT     AS valence,
    (t.audio_features->'tempo')::FLOAT       AS tempo,
    (t.audio_features->'loudness')::FLOAT    AS loudness,
    (t.audio_features->'acousticness')::FLOAT AS acousticness
  FROM dim_track t
  WHERE t.audio_features IS NOT NULL
)
SELECT
  COUNT(*) AS tracks_used,
  ROUND(CORR(popularity, danceability)::NUMERIC, 3) AS corr_danceability,
  ROUND(CORR(popularity, energy)::NUMERIC, 3) AS corr_energy,
  ROUND(CORR(popularity, valence)::NUMERIC, 3) AS corr_valence,
  ROUND(CORR(popularity, tempo)::NUMERIC, 3) AS corr_tempo,
  ROUND(CORR(popularity, loudness)::NUMERIC, 3) AS corr_loudness,
  ROUND(CORR(popularity, acousticness)::NUMERIC, 3) AS corr_acousticness
FROM track_features;

```

BQ4. Which regions or countries have the highest streaming activity for an artist's music?

```

SELECT

```

```

        c.country_name,
        c.region,
        SUM(f.streams) AS total_streams,
        ROUND(100.0 * SUM(f.streams) / SUM(SUM(f.streams)) OVER (), 1) AS pct_of_total
FROM fact_track_performance f
JOIN dim_artist a ON f.artist_key = a.artist_key
JOIN dim_country c ON f.country_key = c.country_key
WHERE a.artist_name = 'Arctic Monkeys'
      AND c.country_code <> 'GLOBAL'
GROUP BY c.country_name, c.region
ORDER BY total_streams DESC;

```

BQ5. How does the performance of an artist's new releases compare with their previous releases?

```

WITH artist_window AS (
    SELECT MAX(d.full_date) AS latest_date
    FROM fact_track_performance f
    JOIN dim_artist a ON f.artist_key = a.artist_key
    JOIN dim_date d ON f.date_key = d.date_key
    WHERE a.artist_name = 'Arctic Monkeys'
),
first_30_days AS (
    SELECT
        t.track_id,
        t.track_title,
        t.release_date,
        SUM(f.streams) AS streams_first_30d,
        ROUND(AVG(f.chart_position), 1) AS avg_position_first_30d
    FROM fact_track_performance f
    JOIN dim_track t ON f.track_key = t.track_key
    JOIN dim_artist a ON f.artist_key = a.artist_key
    JOIN dim_date d ON f.date_key = d.date_key
    WHERE a.artist_name = 'Arctic Monkeys'
          AND d.full_date BETWEEN t.release_date AND t.release_date + 29
    GROUP BY t.track_id, t.track_title, t.release_date
)
SELECT
    CASE WHEN s.release_date > w.latest_date - INTERVAL '12 months'
        THEN 'new release' ELSE 'historical' END AS release_group,
    COUNT(*) AS tracks,
    ROUND(AVG(s.streams_first_30d)) AS avg_streams_first_30d,
    ROUND(AVG(s.avg_position_first_30d), 1) AS avg_chart_position
FROM first_30_days s
CROSS JOIN artist_window w
GROUP BY release_group
ORDER BY release_group DESC;

```